

Newton School of Technology

17

Finding the Path: Link-State & Distance-Vector Routing

Dijkstra and Bellman-Ford applied to routers

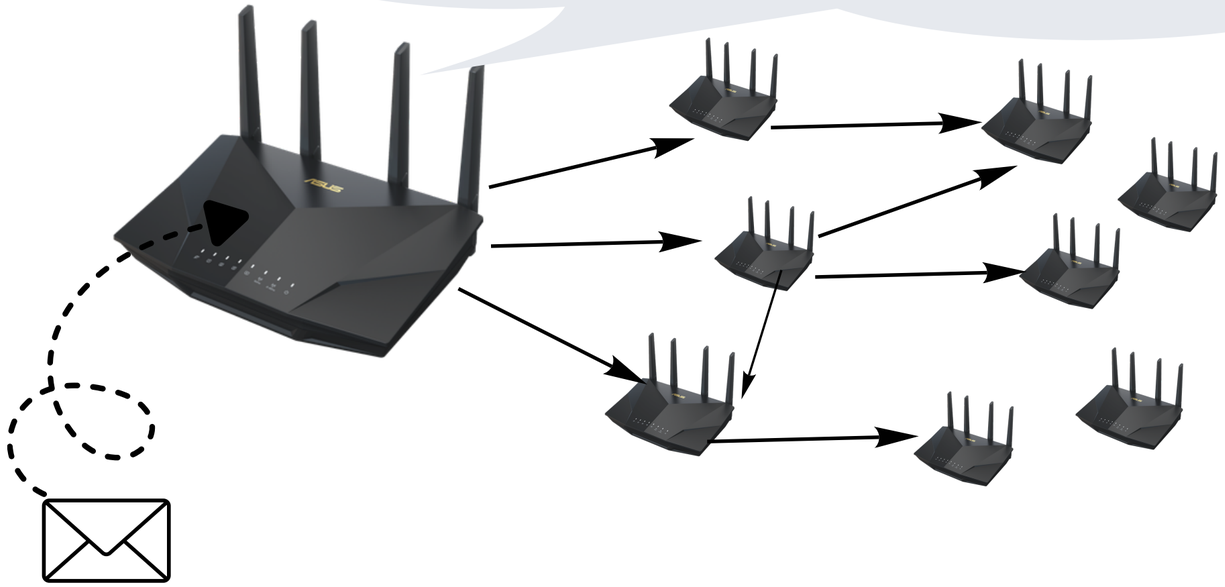
Computer Networks

Join the lecture online on your dashboard

Recap

Newton School of Technology

How do we Know where to forward next?



Destination	NextHop	Interface
12.10.0.1	12.1.1.2	EH1/0/0
12.11.0.1	12.2.2.2	EH2/0/0
12.12.0.1	12.3.3.2	EH3/0/0

Routing Table

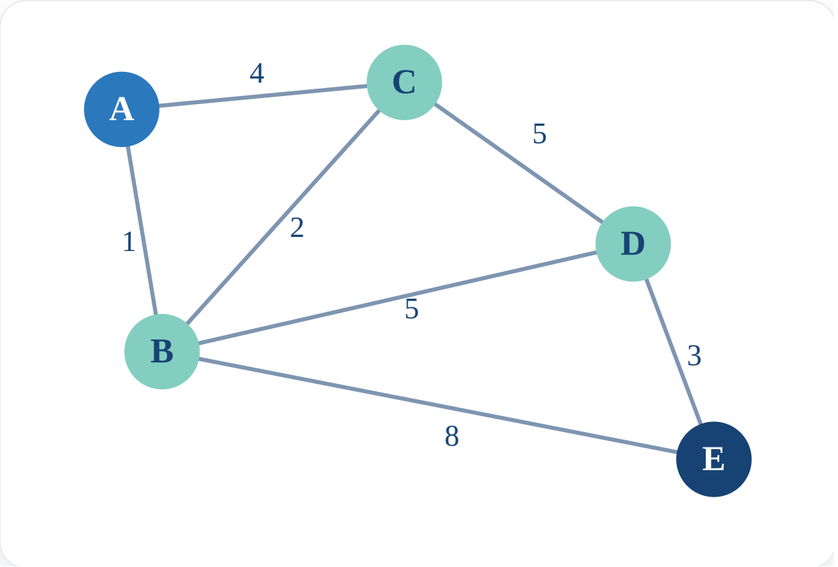
Who Fill these tables?

70 minutes: from "who fills the table" to Understanding thats how they route

Warm-Up: You Already Know This

Shortest path from A to E

- **Route tables (Lecture 15)** decide the next hop, but who fills them across thousands of routers?
- **Algorithms fill them running continuously, on every router.**
- You've written Dijkstra in DSA. Same algorithm new job.



5-Node Weighted Graph

What's the shortest path from A to E, and what does it cost?

Newton School of Technology

1

Routing as a Shortest-Path Problem

Routers *are nodes.* ***Links*** *are edges.* ***Cost*** *is weight.*

Model the Network as a Graph

Every router needs: A forwarding table

- Nodes = routers, edges = links, weight = cost (delay, hops, or inverse bandwidth)
- Goal: each router finds least-cost paths
- Output: destination → next hop (not full path)
- Two approaches: Global (link-state) vs Decentralised (distance-vector)

Router → graph node

Link → graph edge

7 Delay/hops/bandwidth → edge weight

Network → Graph Translation



Analogy:
Google Maps does exactly this intersections are nodes, roads are edges, live traffic time is the weight.

If cost were "fewest hops" instead of "least delay," would the best path change?

Newton School of Technology

2

Link-State: Everyone Gets the Whole Map

Flood the map, then run Dijkstra — locally, on identical data.

Tracing Dijkstra — from Node A

Newton School of Technology

Same mechanics as your DSA implementation

- Maintain a visited set and tentative distances; start with A = 0, all else ∞
- Repeatedly pick the unvisited node with the minimum tentative distance, then relax its neighbours
- **Result: A→C=1, A→B=3, A→D=4, A→E=7 — via A–C–B–D–E**
- Fast convergence, more messages/CPU per router — this is what OSPF runs (next lecture)

```
// distances from A, min-relax order
visit A → d[A]=0
visit C → d[C]=1 (A-C)
visit B → d[B]=3 (A-C-B)
visit D → d[D]=4 (A-C-B-D)
visit E → d[E]=7 (A-C-B-D-E)
```

Dijkstra Forwarding Table for A

Real-world analogy: planning a road trip with the full state map spread on the table — you can eyeball the globally fastest route because you see everything at once.

Every router runs this on the SAME map, so why would two routers ever disagree?

Newton School of Technology

3

Distance-Vector: Trust Only Your Neighbours

No map. Just gossip — and the Bellman-Ford update rule.

The Bellman–Ford Update

Each router only knows its direct neighbours

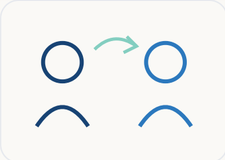
- Every router periodically shares its full distance vector with its neighbours
- **D(dest) = min over neighbour n of [cost(to n) + D_n(dest)]**
- Iterate: each router keeps recomputing as new vectors arrive, until nothing changes
- No global view is ever built, this is what RIP runs (next lecture)

```
// X's distance to Z, before hearing from Y
D_X(Z) = cost(X,Z) = 7

// Y advertises D_Y(Z) = 1, cost(X,Y) = 2
D_X(Z) = min( 7, 2 + 1 ) = 3

X now routes to Z via Y
```

Distance-Vector Update, One Hop



Real-world analogy: asking a local for directions in an unfamiliar town — no map, just their word plus the distance you still have to walk.

X never saw the X–Y–Z path, it just trusted Y's number. What could go wrong with that?

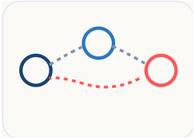
When Good News Travels Fast, Bad News Crawls

The count-to-infinity problem

- Line topology A–B–C, cost 1 each. B and C both route to A, with C going via B
- Link A–B fails. B should announce “A is unreachable”
- **Instead C's stale vector (cost 2 to A) reaches B first, B "believes" C, sets its own cost to 3**
- They keep out-bidding each other upward, round after round, toward infinity

Round	B→A	C→A	
0	3	2	climbing
1	3	4	climbing
2	5	4	climbing
…	…	…	→ ∞

3-Node Count-to-Infinity



Real-world analogy: a broken-telephone rumor, someone repeats stale gossip back to its source, who re-inflates it and passes it on again.

Routers acted on second-hand info without the full map, that's the root cause.

Split Horizon / Poisoned Reverse

One rule that breaks the loop

- **Split horizon: never advertise a route back to the neighbour you learned it from**
- Poisoned reverse: go further, advertise it back as cost ∞, an explicit "don't use me"
- C never tells B "I can reach A", because B is C's only path to A
- Mitigates the simple loop; doesn't solve every topology, but it's the standard first fix

```
B → C: "A costs 1"

C → B: "A costs ∞"  (poisoned)
```

B never mistakes C's route for a real alternative to A.

Poisoned Reverse in Action



Real-world analogy: don't repeat gossip back to the person who told you simple courtesy that also fixes networks.

Split horizon is a rule about what NOT to say — sometimes silence is the fix.

Link-State vs. Distance-Vector

Newton School of Technology

	Link-State (Dijkstra)	Distance-Vector (Bellman-Ford)
Knowledge	Whole network map	Neighbours only
Messages	Flood link-state to everyone	Exchange vectors with neighbours
Convergence	Fast, no loops	Slower, can loop (count-to-infinity)
Scale	CPU/memory heavy per router	Lightweight, but slow to fix failures
Real protocol	**OSPF**	**RIP**

Real-world analogy: link-state is a company-wide org chart everyone can see; distance-vector is office gossip accurate eventually, but only as good as what your neighbour last told you.

BGP, the Internet's inter-domain protocol, is a cousin: path-vector, more in Lecture 18.

Activity time

Newton School of Technology

Five students, five cards, zero global view

- Each of you holds a card listing only your direct-neighbour costs
- Each round: shout your best distance to E to immediate neighbours only, then update your card with the min rule
- **Watch distances ripple outward until they stabilize**
- Then a student steps back (link cut) count-to-infinity happens live, in real time
- Re-run with split horizon watch the loop disappear

Nothing makes decentralised routing click like being a confused router with partial information.

Next Up: The Protocols That Run These Algorithms

Newton School of Technology

Today's algorithms are the engines, Next we will give them a body

1

OSPF

Link-state, runs Dijkstra

2

RIP

Distance-vector, runs Bellman-Ford

3

BGP

Path-vector, the Internet's glue

We'll configure these in Newton packet trqacer and see how AWS Route Tables hide all of it.

Please fill the feedback form.

Thanks for watching!